Project Title:

Intelligent AI Agent for Document Question Answering using LangChain and Retrieval-Augmented Generation (RAG)

Developed By

Chinmaya Sajeevan

Technology

Artificial Intelligence | Natural Language Processing (NLP) | Retrieval Augmented Generation (RAG)

Tools Used

Python, Streamlit, LangChain, Google Gemini API, FAISS Vector Database

Project Type

AI Agent Application with Document Retrieval and Tool Integration

Intelligent AI Agent for Document Question Answering using LangChain and RAG

1. Introduction

Artificial Intelligence agents are systems that can perceive information, reason about it, and perform actions to accomplish tasks. With the advancement of **Large Language Models (LLMs)**, AI agents can now understand natural language and interact with external tools and knowledge sources.

In this project, we develop a **Simple AI Agent using LangChain** that can:

- Answer questions about a **PDF document**
- **Remember conversation context**
- Use an **external tool (Calculator)**
- Handle errors gracefully
- Provide an **interactive web interface using Streamlit**

The system uses **Retrieval Augmented Generation (RAG)** to extract knowledge from a PDF document and combine it with the reasoning capabilities of a **Gemini language model**.

2. Objectives of the Project

The main objectives of this project are:

1. To build an AI agent capable of answering questions from an uploaded PDF document.
2. To implement **Retrieval Augmented Generation (RAG)** for document-based question answering.
3. To enable the agent to **remember conversation context**.
4. To integrate at least **one external tool** (calculator).
5. To implement **error handling mechanisms**.
6. To develop a **user-friendly frontend interface** for interacting with the AI agent.
7. To demonstrate how AI agents can be extended for more complex real-world tasks.

4. System Architecture

The system follows a **Retrieval-Augmented Generation (RAG) architecture combined with a tool-based AI agent**.

System Workflow

1. User uploads a PDF document.
2. The document is processed and split into smaller chunks.
3. Each chunk is converted into vector embeddings.
4. Embeddings are stored in a FAISS vector database.
5. When a user asks a question:
    - The system retrieves relevant document sections.
    - The AI agent processes the query.
    - If necessary, the agent uses external tools (calculator).
6. The Gemini language model generates a final response.
7. The response is displayed to the user.

Architecture Diagram

```
User
 │
 ▼
Streamlit Frontend
 │
 ▼
PDF Upload
 │
 ▼
Document Loader (PyPDFLoader)
 │
 ▼
Text Splitter
 │
 ▼
Embeddings (Google Gemini)
 │
 ▼
Vector Store (FAISS)
 │
```

```
  ▼
LangChain Agent
  ├── PDF Search Tool
  ├── Calculator Tool
  ▼
Gemini LLM
  │
  ▼
Final Response to User
```

4. Technologies Used

Technology	Purpose
Python	Core programming language
Streamlit	Frontend web interface
LangChain	AI agent framework
Gemini API	Large Language Model
FAISS	Vector database
PyPDFLoader	Load PDF documents
RecursiveCharacterTextSplitter	Document chunking
GoogleGenerativeAIEmbeddings	Generate vector embeddings

5. Methodology

The project consists of several key stages.

5.1 PDF Processing

When the user uploads a PDF:

1. The file is temporarily saved
2. The document is loaded using **PyPDFLoader**
3. The document is split into smaller chunks

Chunking is important because:

- LLMs have context limits

- Smaller chunks improve retrieval accuracy

Example:

Chunk Size: 1000 characters
Overlap: 100 characters

5.2 Embedding Generation

Each chunk is converted into **vector embeddings** using:

GoogleGenerativeAIEmbeddings

Embeddings convert text into numerical representations that allow similarity comparisons.

5.3 Vector Database

The embeddings are stored in **FAISS (Facebook AI Similarity Search)**.

FAISS allows:

- Fast similarity search
- Efficient document retrieval

When a query is asked:

similarity_search(query, k=3)

returns the most relevant document chunks.

5.4 AI Agent

The agent is created using **LangChain's create_agent() function**.

The agent:

- Receives the conversation history
- Decides when to use tools
- Calls tools when needed
- Generates final responses using the LLM

5.5 External Tools

The project implements two tools.

Tool 1: PDF Search Tool

This tool retrieves information from the uploaded PDF.

Function:

pdf_search(query)

Steps:

1. Perform similarity search in FAISS
2. Retrieve top relevant chunks
3. Return them to the agent

Tool 2: Calculator Tool

This tool performs mathematical calculations.

Example inputs:

```
144**0.5
50+25*3
```

The calculator safely evaluates expressions.

5.6 Conversation Memory

Conversation context is stored using:

st.session_state.messages

This allows the agent to remember previous interactions.

Example:

User: What is attention?
Assistant: Explanation...

User: Who proposed it?
Assistant understands that **"it" refers to attention**.

5.7 Error Handling

The system handles multiple error cases:

1. Missing API Key

Displays warning.

2. No PDF Uploaded

Tool informs user to upload a document.

3. Gemini API Quota Limit

Displays message when free quota is exhausted.

4. Invalid Math Expression

Calculator returns an error message.

6. Implementation

The system is implemented in **Python using Streamlit**.

Major components:

- Streamlit UI
- Document ingestion pipeline
- Vector database
- LangChain agent
- External tools

7. Key Features

✓ Upload PDF documents
✓ Ask questions about document
✓ Conversational memory
✓ Calculator tool
✓ Error handling
✓ Interactive UI
✓ Retrieval-Augmented Generation

8.The Code – app.py

```
import streamlit as st
import tempfile
import os

# --- 2026 STABLE IMPORTS ---
from langchain_google_genai import GoogleGenerativeAIEmbeddings, ChatGoogleGenerativeAI
from langchain_community.document_loaders import PyPDFLoader
from langchain_community.vectorstores import FAISS
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_core.tools import tool
from langchain.agents import create_agent

# ------------------------
# 1. Session State Init (MUST BE FIRST)
# ------------------------
if "vectorstore" not in st.session_state:
    st.session_state.vectorstore = None
if "messages" not in st.session_state:
    st.session_state.messages = []
```

```python
# 2. UI Setup
# ------------------------
st.set_page_config(page_title="Module 5 Project: AI Agent", layout="wide")
st.title("🚀 AI Agent: RAG + Calculator")

with st.sidebar:
    st.header("Authentication")
    google_api_key = st.text_input("Enter Google API Key:", type="password")
    st.info("Get a free key at [aistudio.google.com](https://aistudio.google.com/)")

    if st.button("Reset App & Clear Chat"):
        st.session_state.messages = []
        st.session_state.vectorstore = None
        st.rerun()

# ------------------------
# 3. Logic: PDF Ingestion
# ------------------------
uploaded_file = st.file_uploader("Upload Project PDF", type="pdf")

if uploaded_file and google_api_key:
    os.environ["GOOGLE_API_KEY"] = google_api_key
    if st.session_state.vectorstore is None:
        try:
            with st.spinner("Building Knowledge Base..."):
                with tempfile.NamedTemporaryFile(delete=False, suffix=".pdf") as tmp:
                    tmp.write(uploaded_file.getvalue())
                    tmp_path = tmp.name

                loader = PyPDFLoader(tmp_path)
                docs = loader.load_and_split(
                    RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=100)
                )
```

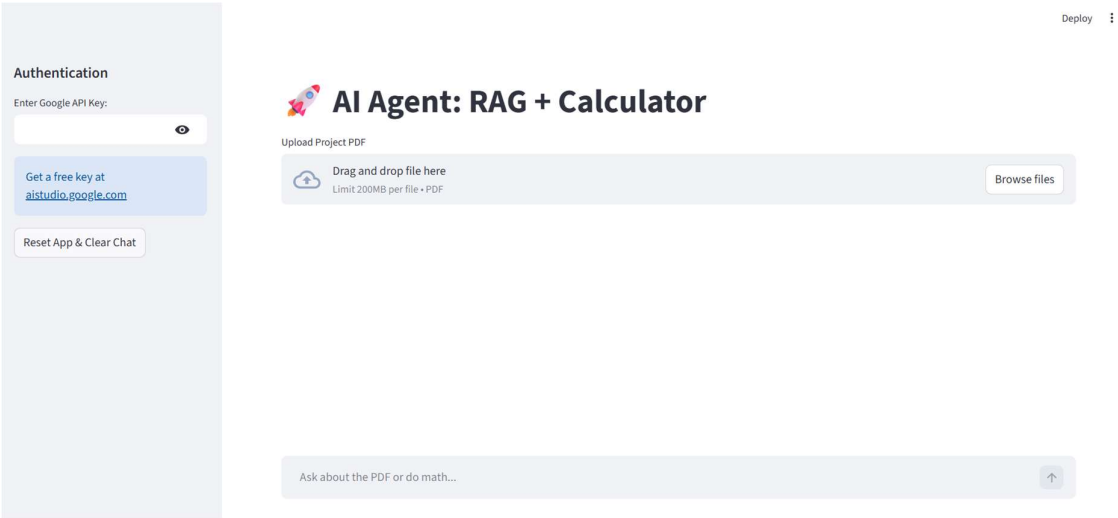
```python
                embeddings = GoogleGenerativeAIEmbeddings(model="models/gemini-embedding-001")
                st.session_state.vectorstore = FAISS.from_documents(docs, embeddings)
                os.remove(tmp_path)
                st.success("✅ PDF Processed for RAG!")
        except Exception as e:
            st.error(f"Upload Error: {e}")

# ------------------------
# 4. Logic: Tools Definition
# ------------------------
@tool
def pdf_search(query: str) -> str:
    """Use this tool to find information inside the uploaded PDF document."""
    if st.session_state.vectorstore is None:
        return "No PDF has been uploaded yet. Please tell the user to upload a document."

    docs = st.session_state.vectorstore.similarity_search(query, k=3)
    return "\n".join([d.page_content for d in docs])
```

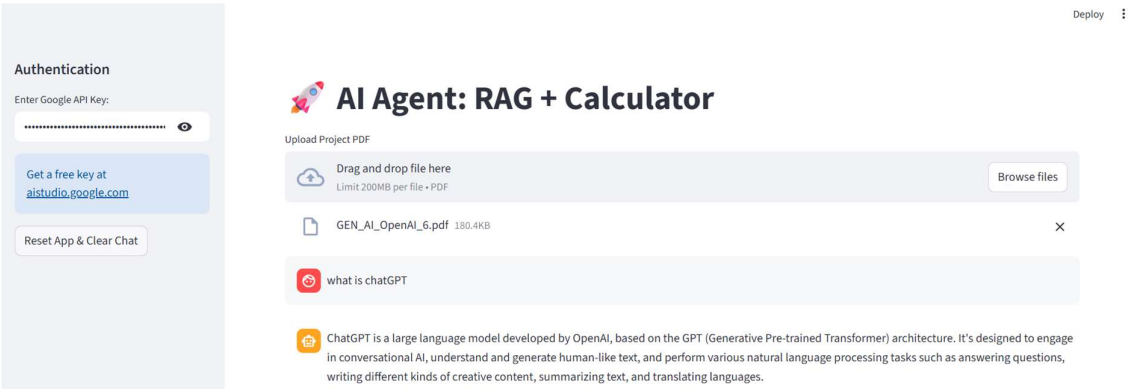
```python
@tool
def calculator(expression: str) -> str:
    """Use this tool for math. Input should be a math expression like '144**0.5 + 50'."""
    try:
        return str(eval(expression, {"__builtins__": None}, {}))
    except:
        return "Error: Please provide a valid math expression."

tools = [pdf_search, calculator]

# ------------------------
# 5. Logic: Chat Interface
# ------------------------
for msg in st.session_state.messages:
    with st.chat_message(msg["role"]):
        st.write(msg["content"])

user_input = st.chat_input("Ask about the PDF or do math...")

if user_input:
    st.session_state.messages.append({"role": "user", "content": user_input})
    with st.chat_message("user"):
        st.write(user_input)

    if not google_api_key:
        st.warning("⚠️ Please enter an API key in the sidebar.")
    else:
        try:
            llm = ChatGoogleGenerativeAI(
                model="gemini-2.5-flash",
                temperature=0,
                api_key=google_api_key
            )
```

```python
            agent = create_agent(
                model=llm,
                tools=tools,
                system_prompt="You are a professional assistant. Use tools for math and PDF search."
            )


            with st.chat_message("assistant"):
                with st.spinner("Thinking..."):

                    response = agent.invoke({"messages": st.session_state.messages})

                    last_message = response["messages"][-1].content
```

```python
            # Fix Gemini structured output
            if isinstance(last_message, list):
                final_answer = ""
                for block in last_message:
                    if isinstance(block, dict) and block.get("type") == "text":
                        final_answer += block.get("text", "")
            else:
                final_answer = str(last_message)

            st.write(final_answer)

            st.session_state.messages.append(
                {"role": "assistant", "content": final_answer}
            )

    except Exception as e:
        if "429" in str(e) or "RESOURCE_EXHAUSTED" in str(e):
            st.error("🔴 **Quota Reached:** You've hit the Gemini free tier limit (20 calls/day). Please wait a minute or try again tomorrow.")
        else:
            st.error(f"Agent Error: {e}")
```

9.The Output





Deploy

Authentication

Enter Google API Key:

••

Get a free key at aistudio.google.com

Reset App & Clear Chat

OpenAI was founded by a group of prominent figures in the tech and AI world. The co-founders include **Elon Musk, Sam Altman, Greg Brockman, Ilya Sutskever, John Schulman, and Wojciech Zaremba**.

When was it founded?

OpenAI was founded in **December 2015**.

What products did they create?

OpenAI has developed several influential AI products, including:

- **GPT (Generative Pre-trained Transformer) series:** This includes models like GPT-3, GPT-3.5, and GPT-4, which are powerful language models capable of generating human-like text, answering questions, summarizing, and more.
- **ChatGPT:** A conversational AI model built on the GPT architecture, designed for interactive dialogue.
- **DALL-E:** An AI system that can generate images from textual descriptions. DALL-E 2 and DALL-E 3 are later iterations with improved capabilities.

Deploy

Authentication

Enter Google API Key:

••

Get a free key at aistudio.google.com

Reset App & Clear Chat

- **Codex:** The AI model that powers GitHub Copilot, designed to translate natural language into code.
- **Whisper:** An open-source general-purpose speech recognition model.
- **OpenAI Gym:** A toolkit for developing and comparing reinforcement learning algorithms.
- **OpenAI Five:** A team of five neural networks that learned to play Dota 2 at a professional level.

If GPT started in 2018 and GPT-4 came in 2023, how many years difference?

There is a **5-year** difference between 2018 and 2023.

What is 1440.5 + 50?**

1440.5 + 50 = 1490.5

Deploy

Authentication

Enter Google API Key:

••

Get a free key at aistudio.google.com

Reset App & Clear Chat

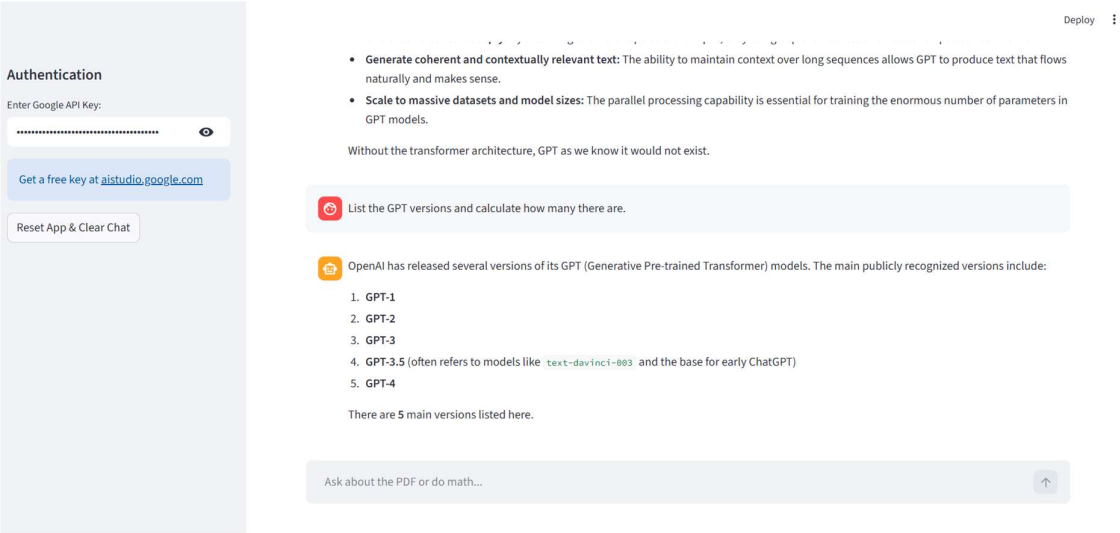
What is the role of transformers in GPT?

Transformers are the foundational neural network architecture that GPT (Generative Pre-trained Transformer) models are built upon. Their role is absolutely crucial to GPT's capabilities, primarily because of two key features:

1. **Self-Attention Mechanism:** This is the core innovation of transformers. It allows the model to weigh the importance of different words in the input sequence when processing each word. For example, when GPT is generating a sentence, the self-attention mechanism helps it understand the context of previous words and how they relate to the current word it's trying to predict. This enables GPT to capture long-range dependencies in text, meaning it can understand relationships between words that are far apart in a sentence or even across multiple sentences. This is a significant improvement over previous architectures like LSTMs or GRUs, which struggled with very long sequences.
2. **Parallel Processing:** Unlike recurrent neural networks (RNNs) which process words sequentially, transformers can process all words in an input sequence simultaneously. This parallelization significantly speeds up training, especially on large datasets and with powerful hardware like GPUs. This efficiency is what made it feasible to train models as massive as GPT-3 and GPT-4 on vast amounts of text data.

In essence, transformers enable GPT models to:

- **Understand context deeply:** By attending to different parts of the input, they can grasp the nuances and relationships between words.
- **Generate coherent and contextually relevant text:** The ability to maintain context over long sequences allows GPT to produce text that flows naturally and makes sense.



10. Advantages of the System

- Enables interactive document analysis
- Improves information retrieval from large documents
- Combines LLM reasoning with external tools
- User-friendly interface
- Easily extendable

11. Limitations

- Limited by LLM API quota
- Works best with text-heavy PDFs
- Requires internet connection for Gemini API
- Complex PDFs (tables/images) may not parse perfectly

12. Design Choices

During the development of this AI agent, several architectural and implementation decisions were made to ensure efficiency, modularity, and scalability.

12.1 Use of LangChain for Agent Development

LangChain was chosen because it provides a structured framework for building AI agents that can interact with tools, external data sources, and large language models.

Advantages of using LangChain:

- Simplifies integration with **LLMs**
- Supports **tool-based reasoning**

- Enables **agent-based decision making**
- Provides built-in support for **RAG pipelines**

This makes LangChain suitable for building intelligent assistants that can perform tasks beyond simple text generation.

12.2 Use of Retrieval Augmented Generation (RAG)

Instead of sending the entire document to the language model, the project uses **Retrieval Augmented Generation (RAG)**.

RAG works by:

1. Converting document text into embeddings.
2. Storing embeddings in a vector database.
3. Retrieving only relevant chunks when a user asks a question.

Benefits:

- Reduces **token usage**
- Improves **response accuracy**
- Handles **large documents efficiently**

This approach is widely used in modern AI assistants.

12.3 Use of FAISS Vector Database

FAISS was selected as the vector database because:

- It provides **fast similarity search**
- It is **lightweight and efficient**
- It works well for **local applications**

FAISS enables the system to quickly retrieve relevant document sections based on semantic similarity.

12.4 Use of Google Gemini Model

The **Gemini Flash model** was selected because:

- It provides **fast responses**
- It supports **tool calling**
- It offers **good reasoning capability**

This model is suitable for building AI assistants that require real-time interaction.

12.5 Use of Streamlit for Frontend

Streamlit was chosen for building the frontend because:

- It allows rapid development of **interactive applications**
- It supports **file uploads**
- It provides **chat-style interfaces**
- It requires minimal frontend coding

This makes it ideal for quickly deploying AI-powered applications.

12.6 Tool-based Agent Architecture

The AI agent was designed to use **external tools**.

Two tools were implemented:

PDF Search Tool

Used to retrieve relevant document sections.

Calculator Tool

Used to solve mathematical expressions.

This design allows the agent to:

- Combine **knowledge retrieval**
- Perform **logical operations**
- Improve response quality

Tool-based agents are more powerful than simple chatbots.

12.7 Session-Based Memory

Conversation memory is maintained using:

st.session_state.messages

This allows the agent to remember previous messages and maintain conversation context.

Benefits:

- Enables **multi-turn conversations**
- Improves **user interaction**
- Helps the agent understand references like “it” or “that”.

13. Extending the Agent for More Complex Tasks

The current system is a simple AI agent, but it can be extended into a much more powerful system.

Below are several possible improvements.

13.1 Multi-Document Knowledge Base

Currently, the system processes only **one PDF document**.

Future improvement:

- Allow multiple documents
- Store them in the same vector database
- Enable cross-document search

This would allow the agent to work like a **knowledge management system**.

13.2 Integration with Web Search

The agent could be extended with a **web search tool**.

Example tools:

- DuckDuckGo Search
- Google Search API
- Wikipedia API

This would allow the agent to retrieve **real-time information from the internet**.

13.3 Advanced Memory Systems

Currently the system stores memory only for the current session.

Future improvements:

- Use **LangChain ConversationBufferMemory**
- Store chat history in a **database**
- Enable **long-term memory**

This would allow the agent to remember user preferences and previous conversations.

13.4 Support for Multiple File Types

The agent could be extended to support additional document types such as:

- Word documents

- Excel files
- CSV datasets
- Web pages

This would transform the agent into a **general document assistant**.

13.5 Data Analysis Tools

The agent could integrate tools for analyzing data.

Examples:

- Python execution tool
- Pandas data analysis
- Chart generation

This would allow the agent to function as a **data analysis assistant**.

13.6 Workflow Automation

The agent could be extended to automate tasks such as:

- Email generation
- Report writing
- Document summarization
- Meeting note extraction

This would enable the agent to assist in **business workflows**.

13.7 Cloud Deployment

The system could be deployed using:

- Streamlit Cloud
- Docker
- AWS
- Hugging Face Spaces

This would allow the application to be accessible globally.

13.8 Multi-Agent Systems

For complex tasks, multiple agents could be created.

Examples:

- Research agent
- Data analysis agent
- Code generation agent

These agents could collaborate to solve large problems.

14. Summary

The design choices in this project focus on:

- **Modularity**
- **Efficiency**
- **Scalability**
- **Ease of development**

By using LangChain, RAG architecture, and tool-based agents, the system becomes flexible and extensible. With further enhancements such as multi-document support, web search, and advanced memory systems, this AI agent can evolve into a powerful intelligent assistant.

15. How to Run the Project

Install dependencies

pip install streamlit langchain langchain-google-genai langchain-community faiss-cpu pypdf

Run the application

streamlit run app.py

Steps

1. Open the Streamlit interface
2. Enter Google API Key
3. Upload PDF
4. Ask questions
5. Use calculator if needed

16. Conclusion

This project demonstrates how **LangChain agents, LLMs, and vector databases** can be combined to create intelligent AI systems.

The developed AI agent can:

- Understand and answer questions about documents
- Maintain conversational context
- Use external tools to enhance its capabilities

Such systems represent the future of **AI-powered knowledge assistants** and can be applied in domains like:

- Research
- Education
- Customer support
- Enterprise knowledge management